

# Database Schema Overview

---

This document outlines the database schema for a system composed of several microservices, including Authentication, User Profiles, Club Management, Event Management, Finance, Notifications, and Reporting.

---

## 1. Auth Service

---

The `auth` schema handles user authentication, registration, and password management.

```
CREATE SCHEMA IF NOT EXISTS auth;
```

```
CREATE TABLE auth.users (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  email TEXT NOT NULL UNIQUE,  
  password_hash TEXT NOT NULL,  
  full_name TEXT NOT NULL,  
  role TEXT NOT NULL CHECK(role IN ('USER', 'SYSTEM_ADMIN', 'GUEST')),  
  is_deleted BOOLEAN NOT NULL DEFAULT FALSE,  
  last_login_at TIMESTAMPTZ,  
  password_changed_at TIMESTAMPTZ,  
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),  
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now()  
);
```

```
CREATE TABLE auth.refresh_tokens (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id UUID NOT NULL REFERENCES auth.users(id) ON DELETE CASCADE,  
  token TEXT NOT NULL UNIQUE,  
  expires_at TIMESTAMPTZ NOT NULL,  
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()  
);  
CREATE INDEX idx_auth_refresh_user ON auth.refresh_tokens(user_id);  
CREATE INDEX idx_auth_refresh_expires ON auth.refresh_tokens(expires_at);
```

```
CREATE TABLE auth.password_reset_tokens (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id UUID NOT NULL REFERENCES auth.users(id) ON DELETE CASCADE,  
  token TEXT NOT NULL UNIQUE,  
  expires_at TIMESTAMPTZ NOT NULL,  
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()  
);
```

```
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
user_id UUID NOT NULL REFERENCES auth.users(id) ON DELETE CASCADE,
token TEXT NOT NULL UNIQUE,
expires_at TIMESTAMPTZ NOT NULL,
created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX idx_auth_prt_user ON auth.password_reset_tokens(user_id);
CREATE INDEX idx_auth_prt_expires ON auth.password_reset_tokens(expires_at);
```

---

## 2. User Service

---

The `user_service` schema stores user profile information, separate from authentication details.

```
CREATE SCHEMA IF NOT EXISTS user_service;

CREATE TABLE user_service.profiles (
  user_id UUID PRIMARY KEY REFERENCES auth.users(id) ON DELETE CASCADE,
  avatar_url TEXT,
  phone TEXT,
  bio TEXT,
  date_of_birth DATE,
  address TEXT,
  social_links JSONB,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX idx_profiles_phone ON user_service.profiles(phone);
```

---

## 3. Club Service

---

The `club_service` schema manages clubs, their memberships, recruitment processes, and announcements.

```
CREATE SCHEMA IF NOT EXISTS club_service;
```

```
CREATE TABLE club_service.clubs (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  name TEXT NOT NULL,  
  description TEXT,  
  type TEXT,  
  size INT,  
  metadata JSONB,  
  logo_url TEXT,  
  website_url TEXT,  
  status TEXT NOT NULL DEFAULT 'ACTIVE' CHECK(status IN ('ACTIVE', 'INACTIVE', '  
  created_by UUID NOT NULL REFERENCES auth.users(id),  
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),  
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now()  
);  
CREATE INDEX idx_clubs_name ON club_service.clubs USING gin(to_tsvector('engli
```

```
CREATE TABLE club_service.memberships (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  club_id UUID NOT NULL REFERENCES club_service.clubs(id) ON DELETE CASCADE,  
  user_id UUID NOT NULL REFERENCES auth.users(id) ON DELETE CASCADE,  
  role TEXT NOT NULL CHECK(role IN ('MANAGER', 'MEMBER')),  
  joined_at TIMESTAMPTZ NOT NULL DEFAULT now(),  
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),  
  UNIQUE (club_id, user_id)  
);  
CREATE INDEX idx_memberships_club ON club_service.memberships(club_id);  
CREATE INDEX idx_memberships_user ON club_service.memberships(user_id);
```

```
CREATE TABLE club_service.recruitment_rounds (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  club_id UUID NOT NULL REFERENCES club_service.clubs(id) ON DELETE CASCADE,  
  title TEXT NOT NULL,  
  description TEXT,  
  criteria TEXT,  
  review_criteria JSONB,  
  quota INT,  
  start_at TIMESTAMPTZ NOT NULL,  
  end_at TIMESTAMPTZ NOT NULL,  
  status TEXT NOT NULL CHECK(status IN ('OPEN', 'CLOSED', 'CANCELLED')),  
  created_by UUID REFERENCES auth.users(id),
```

```

    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX idx_recruitment_club ON club_service.recruitment_rounds(club_id);

CREATE TABLE club_service.applications (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    round_id UUID NOT NULL REFERENCES club_service.recruitment_rounds(id) ON DEL
    user_id UUID NOT NULL REFERENCES auth.users(id) ON DELETE CASCADE,
    application_data JSONB NOT NULL DEFAULT '{}',
    attachment_url TEXT,
    status TEXT NOT NULL CHECK(status IN ('PENDING', 'APPROVED', 'REJECTED')),
    submitted_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    review_score INT CHECK(review_score BETWEEN 0 AND 100),
    review_comments TEXT,
    reviewed_by UUID REFERENCES auth.users(id),
    reviewed_at TIMESTAMPTZ,
    updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    UNIQUE (round_id, user_id)
);
CREATE INDEX idx_applications_round ON club_service.applications(round_id);
CREATE INDEX idx_applications_user ON club_service.applications(user_id);
CREATE INDEX idx_applications_status ON club_service.applications(status);

CREATE TABLE club_service.announcements (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    club_id UUID NOT NULL REFERENCES club_service.clubs(id) ON DELETE CASCADE,
    created_by UUID NOT NULL REFERENCES auth.users(id),
    title TEXT NOT NULL,
    body TEXT NOT NULL,
    attachment_url TEXT,
    valid_until TIMESTAMPTZ,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX idx_announcements_club ON club_service.announcements(club_id);

```

---

## 4. Event Service

---

The `event_service` schema manages event details, organizers, registrations, interests, and tasks.

```
CREATE SCHEMA IF NOT EXISTS event_service;
```

```
CREATE TABLE event_service.events (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  club_id UUID NOT NULL REFERENCES club_service.clubs(id) ON DELETE CASCADE,  
  title TEXT NOT NULL,  
  description TEXT,  
  location TEXT,  
  start_at TIMESTAMPTZ NOT NULL,  
  end_at TIMESTAMPTZ NOT NULL,  
  fee NUMERIC(10,2) DEFAULT 0,  
  status TEXT NOT NULL CHECK(status IN ('DRAFT', 'PUBLISHED', 'CANCELLED')),  
  progress INT DEFAULT 0,  
  max_attendees INT,  
  image_url TEXT,  
  created_by UUID REFERENCES auth.users(id),  
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),  
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now()  
);  
CREATE INDEX idx_events_club ON event_service.events(club_id);  
CREATE INDEX idx_events_start ON event_service.events(start_at);
```

```
CREATE TABLE event_service.organizers (  
  event_id UUID NOT NULL REFERENCES event_service.events(id) ON DELETE CASCADE  
  user_id UUID NOT NULL REFERENCES auth.users(id),  
  PRIMARY KEY (event_id, user_id)  
);
```

```
CREATE TABLE event_service.registrations (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  event_id UUID NOT NULL REFERENCES event_service.events(id) ON DELETE CASCADE  
  user_id UUID NOT NULL REFERENCES auth.users(id),  
  ticket_id UUID NOT NULL UNIQUE,  
  payment_id UUID REFERENCES finance_service.transactions(id),  
  ticket_url TEXT,  
  qr_code_url TEXT,  
  status TEXT NOT NULL CHECK(status IN ('REGISTERED', 'CANCELLED', 'ATTENDED')),  
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),  
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now())
```

```

);
CREATE INDEX idx_regs_event ON event_service.registrations(event_id);
CREATE INDEX idx_regs_user ON event_service.registrations(user_id);
CREATE INDEX idx_regs_payment ON event_service.registrations(payment_id);

CREATE TABLE event_service.event_interests (
    event_id UUID NOT NULL REFERENCES event_service.events(id) ON DELETE CASCADE
    user_id UUID NOT NULL REFERENCES auth.users(id),
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    PRIMARY KEY (event_id, user_id)
);
CREATE INDEX idx_interests_event ON event_service.event_interests(event_id);

CREATE TABLE event_service.event_tasks (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    event_id UUID NOT NULL REFERENCES event_service.events(id) ON DELETE CASCADE
    title TEXT NOT NULL,
    description TEXT,
    status TEXT NOT NULL CHECK(status IN ('TODO', 'IN_PROGRESS', 'DONE')),
    assignee_id UUID REFERENCES auth.users(id),
    due_date TIMESTAMPTZ,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX idx_tasks_event ON event_service.event_tasks(event_id);
CREATE INDEX idx_tasks_assignee ON event_service.event_tasks(assignee_id);

```

---

## 5. Finance Service

---

The `finance_service` schema handles financial transactions and expense requests related to clubs and events.

```

CREATE SCHEMA IF NOT EXISTS finance_service;

CREATE TABLE finance_service.transactions (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    club_id UUID NOT NULL REFERENCES club_service.clubs(id),
    event_id UUID REFERENCES event_service.events(id),

```

```

user_id UUID REFERENCES auth.users(id),
amount NUMERIC(12,2) NOT NULL,
currency TEXT NOT NULL DEFAULT 'VND',
reference_code TEXT,
description TEXT,
type TEXT NOT NULL CHECK(type IN ('CONTRIBUTION', 'FEE', 'REFUND')),
status TEXT NOT NULL CHECK(status IN ('PENDING', 'COMPLETED', 'FAILED')),
transaction_date TIMESTAMPTZ NOT NULL DEFAULT now(),
created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX idx_txn_club ON finance_service.transactions(club_id);
CREATE INDEX idx_txn_event ON finance_service.transactions(event_id);

CREATE TABLE finance_service.expense_requests (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  club_id UUID NOT NULL REFERENCES club_service.clubs(id),
  event_id UUID REFERENCES event_service.events(id),
  requester_id UUID NOT NULL REFERENCES auth.users(id),
  amount NUMERIC(12,2) NOT NULL,
  currency TEXT NOT NULL DEFAULT 'VND',
  description TEXT NOT NULL,
  status TEXT NOT NULL CHECK(status IN ('SUBMITTED', 'APPROVED', 'DENIED')),
  approver_comments TEXT,
  submitted_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  reviewed_by UUID REFERENCES auth.users(id),
  reviewed_at TIMESTAMPTZ,
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX idx_exp_req_club ON finance_service.expense_requests(club_id);
CREATE INDEX idx_exp_req_event ON finance_service.expense_requests(event_id);
CREATE INDEX idx_exp_req_reqstr ON finance_service.expense_requests(requester_

```

---

## 6. Notify Service

---

The `notify_service` schema handles sending and tracking notifications to users via different channels.

```
CREATE SCHEMA IF NOT EXISTS notify_service;

CREATE TABLE notify_service.notifications (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL REFERENCES auth.users(id),
  channel TEXT NOT NULL CHECK(channel IN ('EMAIL', 'SMS')),
  template TEXT NOT NULL,
  payload JSONB NOT NULL,
  status TEXT NOT NULL CHECK(status IN ('PENDING', 'SENT', 'FAILED')),
  error_message TEXT,
  retry_count INT NOT NULL DEFAULT 0,
  sent_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX idx_notify_user ON notify_service.notifications(user_id);
CREATE INDEX idx_notify_status ON notify_service.notifications(status);
```

---

## 7. Report Service

---

The `report_service` schema is responsible for storing aggregated daily metrics and financial summaries for clubs.

```
CREATE SCHEMA IF NOT EXISTS report_service;

CREATE TABLE report_service.daily_metrics (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  report_date DATE NOT NULL UNIQUE,
  total_users INT NOT NULL,
  total_clubs INT NOT NULL,
  total_events INT NOT NULL,
  total_transactions NUMERIC(14,2) NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX idx_metrics_date ON report_service.daily_metrics(report_date);
```

```
CREATE TABLE report_service.club_financial_summary (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  club_id UUID NOT NULL REFERENCES club_service.clubs(id),  
  start_date DATE NOT NULL,  
  end_date DATE NOT NULL,  
  total_revenue NUMERIC(14,2),  
  total_expense NUMERIC(14,2),  
  generated_by UUID REFERENCES auth.users(id),  
  notes TEXT,  
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),  
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),  
  UNIQUE (club_id, start_date, end_date)  
);  
CREATE INDEX idx_club_summary_period ON report_service.club_financial_summary(
```